



Materiały dydaktyczne

Przedmiot: Programowanie obiektowe/Pracownia programowania obiektowego

Opracował: Mirosław Miciak

Ćwiczenie: C17

Tematy: Podział programu na funkcje, Biblioteki języka programowania, Proste i złożone typy danych, Zmienne, Operacje na zmiennych: wejścia, wyjścia, arytmetyczne logiczne, Operacje na typach złożonych, Operatory arytmetyczne, przypisania, porównania, Operatory logiczne i bitowe, Operatory do obsługi łańcuchów, Instrukcja warunkowa i instrukcja wyboru, Instrukcje pętli, Wykorzystanie wybranej biblioteki matematycznej, Wykorzystanie bibliotek algorytmów, Funkcje, Klasa, obiekt, metoda, pole, Projektowanie prostych klas, Zmienne obiektowe, Zakres widoczności pól klasy, Metody klasy, Konstruktor i destruktor, Konstruktor kopiujący, Metody klasy - zakres widoczności, Prosta aplikacja obiektowa: konstruktor, destruktor, Składniki statyczne klasy, Klasy dziedziczone, Klasy zaprzyjaźnione, Dziedziczenie, Hermetyzacja, Polimorfizm, Funkcje zaprzyjaźnione z klasą, Metody do obsługi elementów statycznych klasy, Klasy bazowe i pochodne, Metody wirtualne, Klasy abstrakcyjne, Szablony klas dla prostych typów liczbowych, Obsługa wyjątków, Instrukcja throw, Obsługa błędów wykonania aplikacji, Interfejsy i klasy abstrakcyjne, Przeciążanie operatorów, Wzorce projektowe (Design Patterns), **Kompozycja vs. dziedziczenie** Zasada pojedynczej odpowiedzialności (Single Responsibility Principle), Zasada otwarte-zamknięte (Open-Closed Principle), Zasada podstawienia Liskov (Liskov Substitution Principle), Zasada segregacji interfejsów (Interface Segregation Principle), Zasada odwrócenia zależności (Dependency Inversion Principle), Testowanie jednostkowe klas i metod.

Kompozycja vs. Dziedziczenie w Programowaniu Obiektowym

W programowaniu obiektowym istnieją dwa podstawowe mechanizmy umożliwiające ponowne wykorzystanie kodu i budowanie relacji między klasami: dziedziczenie i kompozycja.

Dziedziczenie – tworzenie hierarchii klas, gdzie klasa potomna (dziecko) dziedziczy właściwości i metody klasy bazowej (rodzica).

Kompozycja – budowanie złożonych obiektów poprzez włączanie innych obiektów jako składowych.

Kiedy stosować dziedziczenie, a kiedy kompozycję?

Dziedziczenie jest odpowiednie, gdy relacja między klasami jest typu "jest" (np. Pies jest Zwierzęciem).

Kompozycja lepiej sprawdza się, gdy relacja jest typu "ma" (np. Samochód ma Silnik).

W poniższych przykładach zostaną zaprezentowane oba podejścia w języku C#.

Przykład 1. Dziedziczenie

```
using System;

// Klasa bazowa (rodzic)
public class Zwierze
{
    public string Gatunek { get; set; }

    public void DajGlos()
    {
    }
```

```

        Console.WriteLine("Zwierzę wydaje dźwięk...");
    }
}

// Klasa pochodna (dziecko) - relacja "Pies JEST Zwierzęciem"
public class Pies : Zwierze
{
    public void Szczekaj()
    {
        Console.WriteLine("Hau hau!");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Pies pies = new Pies();
        pies.Gatunek = "Canis lupus familiaris";
        pies.DajGlos(); // Metoda z klasy bazowej
        pies.Szczekaj(); // Metoda z klasy pochodnej
    }
}

```

Komentarz:

Klasa Pies dziedziczy po Zwierze, więc może używać właściwości i metod klasy bazowej.
Jest to przykład relacji "jest".

Przykład 2. Kompozycja

```

using System;

// Klasa reprezentująca silnik
public class Silnik
{
    public double Pojemnosc { get; set; }
    public void Uruchom()
    {
        Console.WriteLine("Silnik został uruchomiony!");
    }
}

// Klasa reprezentująca samochód - relacja "Samochód MA Silnik"
public class Samochod
{
    private Silnik _silnik; // Kompozycja - silnik jest częścią samochodu

    public Samochod(double pojemnoscSilnika)
    {
        _silnik = new Silnik { Pojemnosc = pojemnoscSilnika };
    }

    public void Jedz()
    {
        _silnik.Uruchom();
        Console.WriteLine("Samochód jedzie!");
    }
}

class Program
{
    static void Main(string[] args)
    {
        Samochod samochod = new Samochod(2.0);
        samochod.Jedz();
    }
}

```

Komentarz:

Klasa Samochod zawiera obiekt Silnik (kompozycja).

Jest to przykład relacji "ma".

Zadania do samodzielnego rozwiązania

1. Dziedziczenie

Stwórz hierarchię klas:

- Klasa bazowa Osoba z polami Imie i Nazwisko.
- Klasa pochodna Student, która dodaje pole NrIndeksu.
- Klasa pochodna Wykładowca, która dodaje pole TytułNaukowy.

Przykład użycia:

```
Student student = new Student { Imie = "Jan", Nazwisko = "Kowalski", NrIndeksu = "12345" };  
Wykładowca wykładowca = new Wykładowca { Imie = "Anna", Nazwisko = "Nowak", TytułNaukowy = "dr  
hab." };
```

2. Kompozycja

Stwórz klasę Komputer, która składa się z:

- Procesor (model, taktowanie)
- PamięćRAM (pojemność)
- DyskTwardy (pojemność, typ: HDD/SSD)

Przykład użycia:

```
Komputer komputer = new Komputer(  
    new Procesor { Model = "Intel i7", Taktowanie = 3.4 },  
    new PamięćRAM { Pojemnosc = 16 },  
    new DyskTwardy { Pojemnosc = 512, Typ = "SSD" }  
);
```

Jak kompozycja uratowała giganta IT przed katastrofą

Dziedziczenie w Javie – problemy Oracle'a

W 2010 roku Oracle przejął Sun Microsystems (twórcę Javy) i odziedziczył po nim... ogromną hierarchię klas w JDK, która z czasem stała się koszmarem utrzymaniowym.

Jednym z najbardziej problematycznych przykładów była klasa `java.util.Stack`, która dziedziczyła po `java.util.Vector`. Dlaczego to był problem?

- `Stack` (stos) powinien mieć tylko metody `push()`, `pop()`, `peek()`.
- `Vector` to lista z dynamicznym rozmiarem, która ma metody `add()`, `remove()`, `get()`.

Efekt?

Programiści mogli robić rzeczy, które łamały logikę stosu, np.:

```
Stack<String> stos = new Stack<>();
stos.push("A");
stos.push("B");
stos.remove(0); // Usuwa element ze środka stosu! To nie powinno być możliwe!
```

To było jak dodanie metody "wyjmij środek szklanki" do projektu kubka – technicznie możliwe, ale sprzeczne z zasadami stosu.

Jak to naprawiono? Kompozycja w nowoczesnej Javie

Zamiast dziedziczenia, twórcy Javy (m.in. Joshua Bloch, autor *"Effective Java"*) zaczęli promować kompozycję.

Przykład: `java.util.Deque` (interfejs kolejki dwustronnej) jest używany przez kompozycję w nowszych implementacjach stosów:

```
Deque<String> stos = new ArrayDeque<>(); // Lepsza alternatywa dla Stack
stos.push("A");
stos.push("B");
// stos.remove(0); // BŁĄD KOMPILACJI – Deque nie pozwala na losowe usuwanie!
```

Dlaczego to działa?

- `ArrayDeque` nie dziedziczy po `Vector`, tylko implementuje `Deque`.
- Metody są kontrolowane – nie ma niechcianych operacji.

Skutki tej zmiany

1. Lepsza kontrola – programiści nie mogą przypadkowo złamać logiki stosu.
2. Wydajność – `ArrayDeque` jest szybszy niż `Stack` (bo nie dziedziczy po przestarzałym `Vector`).
3. Elastyczność – jeśli Oracle zechce zmienić implementację, nie złamie istniejącego kodu.

Co na to *"Effective Java"*?

W słynnej książce Joshua Blocha (byłego głównego architekta Javy) pojawia się zasada:

"Favor composition over inheritance" ("Preferuj kompozycję nad dziedziczeniem").

Dlaczego?

- Dziedziczenie wiąże klasy na śmierć i życie (podklasa zależy od implementacji nadklasy).
- Kompozycja pozwala na wymianę komponentów (np. możesz zastąpić `ArrayDeque` inną kolejką).

Podsumowanie: Lekcja dla programistów

- Dziedziczenie jest jak małżeństwo z kodem – trudno je później unieważnić.

- Kompozycja jest jak umowa o współpracy – możesz zmienić dostawcę, gdy zajdzie potrzeba.
- Czy Java całkiem porzuciła dziedziczenie? Nie! Nadal jest użyteczne, np. w:
- Klasach abstrakcyjnych (np. `AbstractList`).
 - Frameworkach (np. dziedziczenie `Activity` w Androidzie).
- Ale tam, gdzie to możliwe, kompozycja wygrywa.

Czy w Twoim kodzie też są "Stacki", które powinny stać się "Deque"?